If you keep an external reference to an object that is used as a key and use that reference to alter the object, thus changing its hash code, the hash lookup based on that key may not work. You can force the hash to be reindexed by calling its rehash method.

```
arr = [1, 2, 3]
hash = {arr => "value"}
hash[arr] # => "value"
arr[1] = 99
hash      # => {[1, 99, 3]=>"value"}
hash[arr] # => nil
hash.rehash
hash[arr] # => "value"
```

Because strings are very frequently used as hash keys and because string contents are often changed, Ruby treats string keys as a special case. If you use a String object as a hash key, the hash will duplicate the string internally and will use that copy as its key. The copy will be frozen. Any changes made to the original string will not affect the hash.

If you write your own classes and use instances of them as hash keys, you need to make sure that either (a) the hashes of the key objects don't change once the objects have been created or (b) you remember to call the rehash method to reindex the hash whenever a key hash is changed.

Hash Shortcuts

Often, a hash key has the same name as a value in the current context, and you want to assign that value to that key: {x: x, y: y}. You may omit the value when creating a hash literal:

```
x = 10
y = 20
{x:, y:}  # => {:x=>10, :y=>20}
```

The key can be a local variable or a method that takes no arguments.

Symbols

A Ruby symbol is an identifier similar to a string, but optimized for fast lookup. If the symbol you want is an identifier—no spaces, no unusual characters–you construct the symbol for a name by preceding the name with a colon. You can construct the symbol for any other arbitrary string by preceding a string literal with a colon. Substitution occurs in double-quoted strings.

A particular name or string will always generate the same symbol and the same internal object, regardless of how that name is used within the program. You can also use the %s delimited notation to create a symbol.

```
:Object
:my_variable
:"Ruby rules"
a = "cat"
:'catsup'  # => :catsup
:"#{a}sup" # => :catsup
:'#{a}sup' # => :"\#{a}sup"
```